Reviewer Pack — Minimal Frobenius-Schur Indicator and Communication Complexi…

Entry 6a68d29181d9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 18:35:19 UTC

Minimal Frobenius-Schur Indicator and Communication Complexity Rank Inequality

Entry ID: `6a68d29181d9` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-05 18:35:19 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory (Frobenius-Schur Indicators) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

> For every n × n binary matrix A, the Frobenius-Schur indicator I_F(A) of A is linearly correlated with its communication complexity rank r_A such that I_F(A) = O(r_A^(2/3)) and I_F(A) = Ω(n^{1/3}).

Rationale (proposer's reasoning):

> The Frobenius-Schur indicator captures the non-abelian aspects of group representations, while communication complexity measures the complexity of distributed computation. Their correlation may reveal a structural connection between representation theory and distributed algorithms.

Taxonomy category: `Representation Theory × Communication Complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `4921b9fc12b7a9d5`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all generated n × n binary matrices with n ≤ 40, both (i) the ratio of Frobenius-Schur indicator to r_A^(2/3) is less than or equal to a constant C1 and (ii) the ratio of n^(1/3) to the Frobenius-Schur indicator is less than or equal to a constant C2. The conjecture is falsified if either condition is not met for any matrix.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	0.80	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"Frobenius-Schur indicator" AND "communication complexity rank"` - `"matrix rank" IN "Representation Theory" AND "Communication Complexity"` - `"linear correlation" BETWEEN "Frobenius-Schur indicator" AND "communication complexity"`

Top relevant hits considered: - [http://arxiv.org/abs/1612.03393v6] Exact recovery low-rank matrix via transformed affine matrix rank minimization - [http://arxiv.org/abs/0906.3499v4] Convergence of fixed-point continuation algorithms for matrix rank minimization - [http://arxiv.org/abs/1902.06476v1] Sylvester matrix rank functions on crossed products - [http://arxiv.org/abs/math-ph/9805020v2] Chern–Simons Terms as an Example of the Relations Between Mathematics and Physics - [http://arxiv.org/abs/0801.2027v1] Strong Linear Correlation Between Eigenvalues and Diagonal Matrix Elements - [http://arxiv.org/abs/2303.03958v1] The Linear Correlation of P and NP

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def frobenius_schur_indicator(A):
        n = len(A)
        char_table = [[0] * n for _ in range(n)]
        orthonormal_basis = [[0] * n for _ in range(n)]

        # Fill character table and orthonormal basis (simplified example)
        for i in range(n):
            char_table[i][i] = 1
            orthonormal_basis[i][i] = 1

        I_F = sum(sum(A[i][j] * char_table[i][k] * orthonormal_basis[j][k] for k in range(n)) for
        return I_F

    def communication_complexity_rank(A):
        n = len(A)
        rank = 0
        # Simplified example: rank is the number of non-zero rows/columns
        for row in A:
            if any(row):
```

```
                rank += 1
        return rank

    results = []
    for n in [5, 10, 15, 20, 30, 40]:
        for _ in range(5):  # Ensure at least 30 instances per seed
            A = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]
            I_F = frobenius_schur_indicator(A)
            r_A = communication_complexity_rank(A)

            if r_A == 0:
                continue

            ratio_1 = Fraction(I_F, r_A ** (2/3))
            ratio_2 = Fraction(n ** (1/3), I_F)

            results.append((n, I_F, r_A, ratio_1, ratio_2))

    if not results:
        return {
            "metric_name": "Frobenius-Schur Indicator and Communication Complexity Rank",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": "No valid matrices generated"
        }

    n_max = max(n for n, _, _, _, _ in results)
    metric_value = sum(ratio_1 + ratio_2 for _, _, _, ratio_1, ratio_2 in results) / len(results)
    conjecture_holds = all(ratio_1 <= 10 and ratio_2 <= 10 for _, _, _, ratio_1, ratio_2 in result
    counterexample = "" if conjecture_holds else "Mapping undefined"

    return {
        "metric_name": "Frobenius-Schur Indicator and Communication Complexity Rank",
        "metric_value": metric_value,
        "instances_tested": len(results),
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41,

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    else:
```

```
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample='Mapping undefined' first_failing_seed={first_fai
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c4d74fca.py", line 87, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c4d74fca.py", line 53, in run_trial
    ratio_1 = Fraction(I_F, r_A ** (2/3))
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^
   File "/usr/lib/python3.12/fractions.py", line 277, in __new__
    raise TypeError("both arguments should be "
TypeError: both arguments should be Rational instances
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed during execution, which prevents us from verifying the conjecture's conditions. | next: Review and debug the test code to ensure it can run successfully and produce results.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	19892
2	propose	ollama_remote	glm4:latest	0	0	16043
3	propose	ollama_remote	glm4:latest	0	0	11433
4	propose	ollama_remote	glm4:latest	0	0	20051
5	preregistration	ollama_remote	glm4:latest	0	0	10124
6	novelty	ollama_remote	glm4:latest	0	0	8539
7	novelty	ollama_remote	glm4:latest	0	0	9829
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18703
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14722
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13174
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11366
12	judge	ollama_remote	glm4:latest	0	0	12399

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 166276 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/6a68d29181d9.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/6a68d29181d9.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/6a68d29181d9.tar.gz` (if generated)